

WEEKLY CYBERSECURITY TOOLS TRACK

Netcat (nc)

File Transfer & Two-Way Communication

Hands-on lab practicals using Netcat between a Kali Linux attacker VM and a Metasploitable2 target VM, run over a private VirtualBox host-only network.

Prepared by

Emmanuel Aidoo

Lab environment: Kali Linux (attacker) · Metasploitable2 (target) · Oracle VirtualBox

Table of Contents

Project Overview 3

1. File Transfer with Netcat 4

What I Learned..... 4

2. Two-Way Communication (Chat) with Netcat..... 6

What I Learned..... 6

Key Takeaways..... 8

Project Overview

Netcat (nc) is a lightweight command-line networking utility often described as the "TCP/IP Swiss Army knife." It reads and writes data across network connections using TCP or UDP, which makes it useful for banner grabbing, port scanning, file transfer, and building simple client/server communication channels.

This module documents two beginner practicals carried out across two virtual machines on the same VirtualBox host-only network: a Metasploitable2 VM acting as one endpoint (192.168.56.102) and a Kali Linux VM acting as the other endpoint (192.168.56.101). The goal was to understand Netcat's listener/client model (-l to listen, and a plain nc <ip> <port> to connect) before moving on to more advanced uses such as reverse shells and port scanning in later modules.

ENVIRONMENT

Setting	Detail
Attacker / client VM	Kali Linux — 192.168.56.101
Target / server VM	Metasploitable2 — 192.168.56.102
Network	VirtualBox host-only adapter (192.168.56.0/24)
Tool	Netcat (nc)

1. File Transfer with Netcat

Netcat can move a file between two hosts with no dedicated file-transfer protocol: one side listens and redirects the incoming stream into a file, and the other side connects and feeds a file into the stream. This practical sent a text file from the Kali VM to the Metasploitable2 VM over port 4444.

COMMANDS USED

```
# On Metasploitable2 (listener / receiver) – 192.168.56.102
$ nc -lvp 4444 >received.txt

# On Kali (client / sender) – 192.168.56.101
$ nc 192.168.56.102 4444 <send.txt
```

What I Learned

The receiving side is set up first: `nc -lvp 4444` tells Netcat to listen (`-l`), print connection details verbosely (`-v`), stay on port 4444 (`-p 4444`), and the shell redirection `>received.txt` writes anything received on that socket straight into a file instead of the screen. The sending side is a plain client connection, `nc <target-ip> <port>`, with `<send.txt` redirecting the local file's contents into the socket as the payload.

As soon as the client connects, Netcat has no built-in concept of "end of file" for a transfer. It simply streams bytes until the input (or the connection) ends, which is why this pattern works for arbitrarily large files as well as small text files. On the listener side, the terminal reported an inverse (reverse-DNS) host lookup failure for the connecting address; this is expected in an isolated lab network with no DNS server and does not affect the transfer.

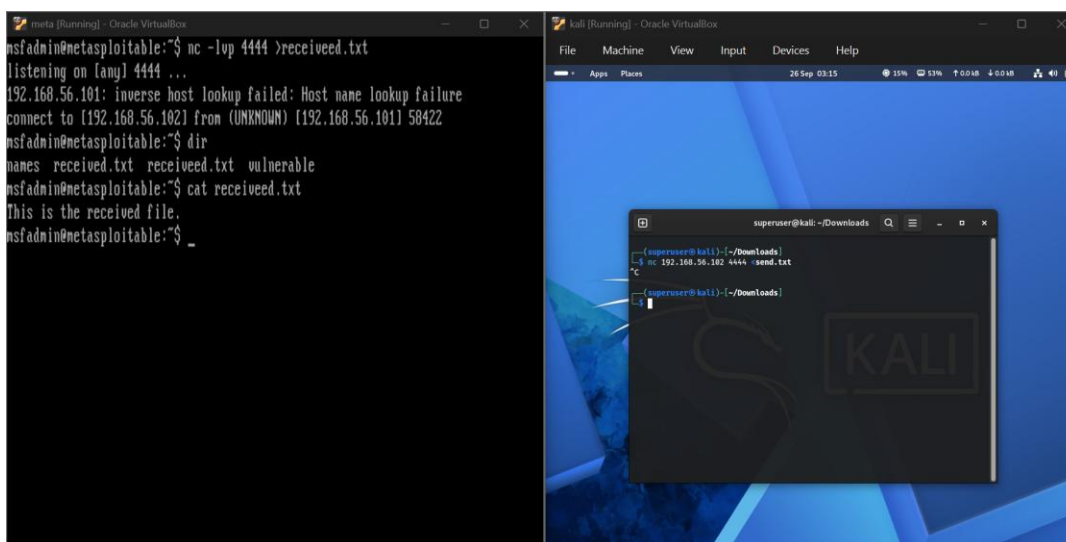


Figure 1 — Kali sending `send.txt` to the Metasploitable2 listener on port 4444; the received file is verified with `dir` and `cat` on the target.

RESULT SUMMARY

Item	Observation
------	-------------

Listener command	nc -lvp 4444 >receiveed.txt (Metasploitable2)
Sender command	nc 192.168.56.102 4444 <send.txt (Kali)
Connection reported	connect to [192.168.56.102] from (UNKNOWN) [192.168.56.101] 58422
File received	receiveed.txt — content: "This is the received file."
Verification	dir listed the new file; cat receiveed.txt confirmed its contents
My Observation I mistyped the output filename as receiveed.txt (double "e") in the listener command, and the transfer still completed without error — Netcat doesn't validate filenames, it just writes to whatever path the shell redirection gives it. I noticed the listener never terminates on its own after the file is fully received instead I had to confirm completion by checking the file and then close the listener with Ctrl+C.	

2. Two-Way Communication (Chat) with Netcat

Beyond files, a plain Netcat connection is just a raw TCP pipe, so typing text on either end sends it live to the other side. This practical set up a simple two-way chat between the Kali and Metasploitable2 VMs, with the listener role swapped to Kali this time.

COMMANDS USED

```
# On Kali (listener) – 192.168.56.101
$ nc -n -lvp 4444

# On Metasploitable2 (client) – 192.168.56.102
$ nc 192.168.56.101 4444
```

What I Learned

Once the client connects, Netcat keeps the TCP connection open in the foreground on both ends, and stdin/stdout are simply piped across the socket so a full-duplex chat happens as soon as both sides start typing, with no extra chat protocol involved. This time the `-n` flag was added on the listener side, which tells Netcat to skip DNS/reverse-DNS resolution entirely; because the earlier file-transfer practical logged a host-lookup failure for the same reason, adding `-n` here suppressed that message and connected slightly faster.

The conversation shown below ("Hi / Hello / How are you doing? / I am good / Yourself? / I am doing amazing, thanks for asking.") appears identically on both terminals in real time, which is the clearest demonstration that Netcat is not doing anything file- or protocol-specific here, it is only relaying raw bytes between two open sockets.

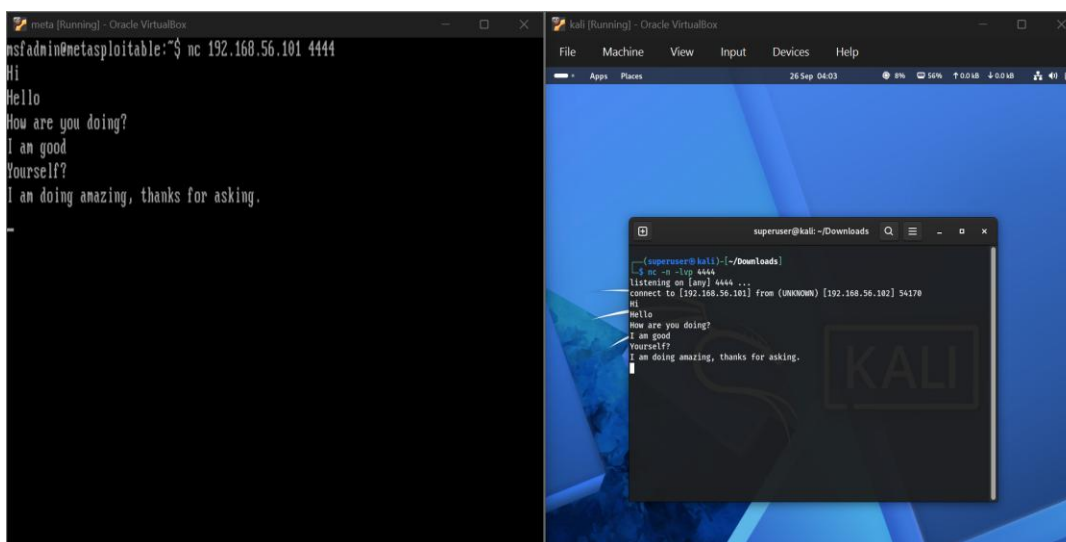


Figure 2 — A live two-way chat over a single Netcat TCP connection, shown mirrored on the Metasploitable2 (left) and Kali (right) terminals.

RESULT SUMMARY

Item	Observation
Listener command	nc -n -lvp 4444 (Kali)
Client command	nc 192.168.56.101 4444 (Metasploitable2)
Connection reported	connect to [192.168.56.101] from (UNKNOWN) [192.168.56.102] 54170
Behaviour	Full-duplex, real-time identical text exchange on both terminals
Flag change	-n added on the listener to skip DNS lookups seen in Practical 1
My Observation This practical made it click that a raw Netcat connection has no idea whether it's carrying a chat message, a file, or a shell command. It's all just bytes on a socket. That's exactly why the same tool doubles as a file-transfer utility in Section 1 and a chat channel here, and why (in a later, more advanced module) the same pattern with a shell bound to the socket becomes a reverse shell. The -n flag is a small but useful habit to carry forward: it removes the reverse-DNS delay/error I saw in the first practical whenever the target network has no working DNS.	

Key Takeaways

- Netcat's core model is simple: one side listens with -l (optionally -v for verbose, -p <port> for the port), and the other side connects with a plain nc <ip> <port>.
- Shell redirection (> to write, < to read) is what turns a raw Netcat pipe into a file-transfer tool. Netcat itself has no concept of "file"; it only moves bytes.
- A live Netcat connection is inherently two-way (full-duplex): typing on either end sends data to the other immediately, which is the same mechanism a reverse or bind shell later relies on.
- The -n flag skips DNS/reverse-DNS resolution, which avoids the host-lookup delay/error seen when connecting on a lab network with no DNS server.